VulnerableVault Audit Report

Version 1.0

alva-p.xyz

April 7, 2026

Prepared by: alva-p Lead Auditors: - Alvaro Pineda

Table of Contents

- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
  - Medium
  - Low
  - Informational

Protocol Summary

VulnerableVault is a simple token vault deployed on BattleChain Testnet (Chain 627) as part of the BattleChain Safe Harbor CTF. It consists of two core contracts:

1. **MockToken** (0x50c756fE8934284496da43ab2D27EFd6D36CafF9): An ERC20 token with open minting and a transfer hook system that allows any address to register a callback contract invoked on incoming transfers.
2. **VulnerableVault** (0xB202Db3b2691843cB89fD7Dc804B6cB9a98D1a78): A token vault that allows users to deposit and withdraw MockToken. Users deposit tokens via deposit() and retrieve their full balance via withdrawAll().

The protocol is intentionally vulnerable as part of a BattleChain CTF / white-hat challenge. The primary vulnerability is a reentrancy attack enabled by a CEI (Checks-Effects-Interactions) violation in the vault's withdrawal function, combined with the token's transfer hook mechanism.

Disclaimer

The alva-p team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Table 1: Likelihood

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

- **Commit Hash:** d532f01 (branch: main)
- **Solidity Version:** 0.8.24
- **Framework:** Foundry
- **Network:** BattleChain Testnet (Chain ID 627)
- **Dynamic Testing:** Foundry test suite (1 test, passing – reentrancy PoC confirmed)

Scope

```
./src/
  VulnerableVault.sol
  MockToken.sol
  Attacker.sol
```

Deployed Addresses:

Contract	Address
VulnerableVault	0xB202Db3b2691843cB89fD7Dc804B6cB9a98D1a78
MockToken	0x50c756fE8934284496da43ab2D27EFd6D36CafF9
Attacker	0x4d425De5365f2174Cb8740C2941D4c79c21360E2

Roles

- **Depositor** – Any address that deposits MockToken into the VulnerableVault via deposit().
- **Withdrawer** – Any depositor who calls withdrawAll() to retrieve their tokens.
- **Token Minter** – Any address (no access control on mint()).
- **Hook Registrant** – Any address that registers a transfer hook via setTransferHook().

Executive Summary

The VulnerableVault protocol contains a **critical reentrancy vulnerability** that allows complete drainage of all vault funds in a single transaction. The attack is enabled by the combination of two design flaws: (1) a CEI violation in VulnerableVault::withdrawAll() where the token transfer occurs before the balance is zeroed, and (2) MockToken's unrestricted transfer hook system that provides the reentry vector.

Additionally, MockToken has no access control on its mint() function, allowing unlimited token creation by any address.

The exploit was verified via Foundry testing and **executed on-chain** on BattleChain Testnet (Chain ID 627, Block 4554). 1,000 tokens were drained from the vault in a single transaction, with the attacker needing only a 100-token seed deposit.

Issues found

Table 3: Issues found summary

Severity	Number of issues found
High	1
Medium	2
Low	1
Info	2
Total	6

Findings

High

[H-1] Reentrancy in `VulnerableVault::withdrawAll()` allows complete vault drainage via MockToken transfer hook

Severity: High (Impact: High, Likelihood: High)

Location: src/VulnerableVault.sol, lines 40-51

Description:

The withdrawAll() function violates the Checks-Effects-Interactions (CEI) pattern. It performs the token transfer (Interaction) on line 45 **before** zeroing the caller's balance (Effect) on line 48:

```
function withdrawAll() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "nothing to withdraw");

    // INTERACTION before EFFECT
    TOKEN.transfer(msg.sender, amount); // line 45 — external call

    // EFFECT after INTERACTION
    balances[msg.sender] = 0;           // line 48 — too late

    emit Withdrawn(msg.sender, amount);
}
```

MockToken's transfer() function (line 41-50 of MockToken.sol) checks if the recipient has a registered transfer hook and calls onTokenTransfer() on that hook contract **after** the token transfer completes. This callback provides the reentry vector: the attacker's hook calls withdrawAll() again while balances[msg.sender] still holds the original non-zero value.

The attack chain is:

1. Attacker deploys a contract implementing ITransferHook and registers it via MockToken::setTransferHook().
2. Attacker mints seed tokens (open mint()) and deposits them into the vault.
3. Attacker calls withdrawAll().
4. Vault calls TOKEN.transfer(attacker, amount) – tokens move.
5. MockToken detects the attacker's hook and calls onTokenTransfer().
6. Inside onTokenTransfer(), the attacker calls withdrawAll() again. Since balances[attacker] has not been zeroed yet, the require(amount > 0) check passes.
7. Steps 4-6 repeat recursively until TOKEN.balanceOf(vault) == 0.
8. All the nested calls unwind, and balances[attacker] = 0 executes once – after all funds are already gone.

Impact:

- **Total loss of all vault funds.** Every token deposited by every user is drained.
- The attacker only needs a small seed deposit (e.g., 100 tokens) to drain an arbitrarily large vault.
- Legitimate depositors' recorded balances become unwithdrawable (the vault has no tokens left but balances[user] is still non-zero).
- This was confirmed in the Foundry test: a 1,000-token vault was completely drained with a 100-token seed.

Proof of Concept:

The provided Attacker.sol contract and ReentrancyTest.t.sol test fully demonstrate the exploit. Running forge test −vvv confirms:

```
Vault balance before: 1000 tokens
── Vault drained ──
Vault before:        1000 tokens
Vault after:         0 tokens
Bounty kept:         110 tokens
```

```
Returned to protocol: 990 tokens
```

On-Chain Execution (BattleChain Testnet – Chain ID 627):

The exploit was successfully executed on-chain on April 7, 2026:

Detail	Value
Attacker Contract	0x4d425De5365f2174Cb8740C2941D4c79c21360E2
Deploy TX	0x812e1d18b18204616789d13826b6b8b48413b3b7ea8bb85d390684
Attack TX	0x59f36c54fd31a4c6e104bc3789fdd8f23cafc98cf2afc975df8681da7d
Block	4554
Vault Before	1,000 tokens
Vault After	0 tokens
Attacker Balance	1,100 tokens (1,000 drained + 100 seed)

Recommended Mitigation:

Apply the CEI pattern – zero the balance **before** making the external call. Additionally, add a reentrancy guard as defense in depth.

```
function withdrawAll() external nonReentrant {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "nothing to withdraw");

    // EFFECT first
    balances[msg.sender] = 0;
    emit Withdrawn(msg.sender, amount);

    // INTERACTION second
    TOKEN.transfer(msg.sender, amount);
}
```

Use OpenZeppelin's ReentrancyGuard for the nonReentrant modifier as a safety net. The CEI fix alone is sufficient to prevent this specific attack, but the guard protects against future modifications that might re-introduce interaction-before-effect patterns.

Medium

[M-1] `MockToken::mint()` has no access control – anyone can mint unlimited tokens

Severity: Medium (Impact: High, Likelihood: Medium)

Location: src/MockToken.sol, lines 29-31

Description:

The mint() function is completely unrestricted:

```
function mint(address to, uint256 amount) external {
    _mint(to, amount);
}
```

Any address can call mint() with an arbitrary to address and amount. While this is intentional for the BattleChain tutorial, in any production context this is a critical flaw. An attacker can mint unlimited tokens, deposit them into the vault, and dilute or manipulate the vault's accounting.

This also directly enables the reentrancy attack in [H-1] by allowing the attacker to create seed tokens at zero cost.

Impact:

- Unlimited token inflation.
- Attacker can create tokens to deposit into any protocol integrating with MockToken.
- Combined with the vault's reentrancy, this enables a zero-cost exploit (mint seeds, drain vault, keep everything).

Recommended Mitigation:

Restrict minting to an authorized role:

```
address public owner;

modifier onlyOwner() {
    require(msg.sender == owner, "not owner");
    _;
}

function mint(address to, uint256 amount) external onlyOwner {
```

```
    _mint(to, amount);
}
```

Alternatively, use OpenZeppelin's AccessControl or Ownable pattern.

[M-2] Unrestricted transfer hook registration enables arbitrary callback injection

Severity: Medium (Impact: High, Likelihood: Medium)

Location: src/MockToken.sol, lines 34-37

Description:

The setTransferHook() function allows **any address** to register an arbitrary contract as a callback hook:

```
function setTransferHook(address hook) external {
    transferHooks[msg.sender] = hook;
    emit TransferHookSet(msg.sender, hook);
}
```

This hook is then called inside transfer() (line 44-46) whenever tokens are sent to the registrant:

```
address hook = transferHooks[to];
if (hook != address(0)) {
    ITransferHook(hook).onTokenTransfer(msg.sender, amount);
}
```

This design converts every transfer() call into a potential external call to attacker-controlled code. Any protocol that calls MockToken.transfer() (such as VulnerableVault) is exposed to reentrancy or other callback-based attacks. The hook mechanism also only applies to transfer() but **not** to transferFrom(), creating inconsistent behavior.

Impact:

- Any contract calling MockToken.transfer() to send tokens to an address with a registered hook will execute arbitrary external code.
- This is the enabling mechanism for the reentrancy in [H-1].
- The inconsistency between transfer() (has hooks) and transferFrom() (no hooks) can lead to unexpected behavior.

Recommended Mitigation:

1. Remove the hook mechanism entirely if not needed.
2. If hooks are required, apply them consistently to both transfer() and transferFrom().
3. Consider using ERC-777 or ERC-1363 standards which have established patterns for transfer callbacks with proper reentrancy protections.
4. At minimum, document the reentrancy risk prominently so integrating protocols are aware.

Low

[L-1] `VulnerableVault::withdrawAll()` does not return the withdrawn amount

Severity: Low

Location: src/VulnerableVault.sol, lines 40-51

Description:

The withdrawAll() function does not return the amount withdrawn. Integrating contracts cannot programmatically determine how many tokens were received without checking balances before and after the call.

Recommended Mitigation:

```
function withdrawAll() external returns (uint256 amount) {
    amount = balances[msg.sender];
    require(amount > 0, "nothing to withdraw");
    balances[msg.sender] = 0;
    TOKEN.transfer(msg.sender, amount);
    emit Withdrawn(msg.sender, amount);
}
```

Informational

[I-1] Floating pragma used across all contracts

All contracts use pragma solidity ^0.8.24 instead of a fixed version. Consider locking to a specific version for production deployments:

```
-pragma solidity ^0.8.24;
+pragma solidity 0.8.24;
```

[I-2] Missing `deposit()` return value and no zero-amount check

The deposit() function does not validate that amount > 0. While a zero-amount deposit is harmless, it wastes gas and emits a misleading Deposited event.

Safe Harbor Distribution

The exploit was executed under BattleChain's Safe Harbor protocol. Recovered funds were distributed per the agreement terms:

Recipient	Amount	Percentage
Recovery Address	990 BCDT	90%
Whitehat Bounty	110 BCDT	10%
Total Recovered	**1,100 BCDT**	**100%**

- **Agreement Address:** 0xCE2983b0102A0cE3DF059AAA19ddf003d73F03Da
- **Agreement State:** 3 (UNDER_ATTACK) – approved by DAO